Input. Set of n points in the plane $(x_i, y_i) \quad \forall i \in \{1 \dots n\}$

Output: closest pair of points



Naive algorithm

Compare all $\binom{n}{2}$ pairs and report the min

Time Complexity $O(n^2)$

Consider when points are in $\mathbb{R}$



Alg - ① Sort all the points — $O(n \log n)$
② For each point look at previous and next point (δ_1, δ_2)
③ Pick the overall min

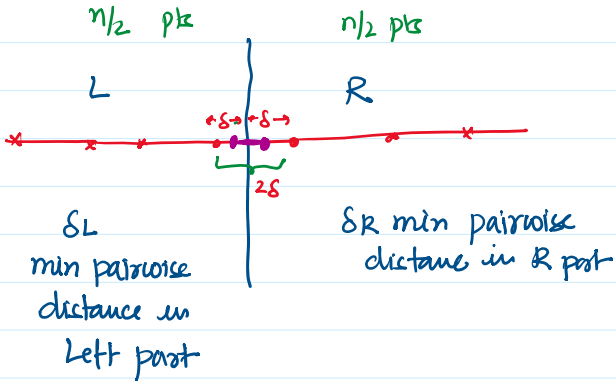
Overall time complexity $O(n \log n)$

Divide and Conquer - (one-dim)

1) Divide the input into two halves (using median Algo)

② Recurse on each part and compute δ_L, δ_R

③ return $\min\{\delta_L, \delta_R, \delta\}$



$\delta = \min\{\delta_L, \delta_R\}$

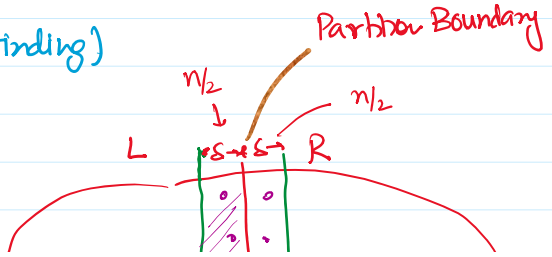
$$T(n) = 2T(n/2) + O(n) + O(1) \leftarrow \text{step 3}$$

↑ Finding median

$$= O(n \log n)$$

This algo doesn't use sorting (except for median finding)

Extend this idea when points are on plane



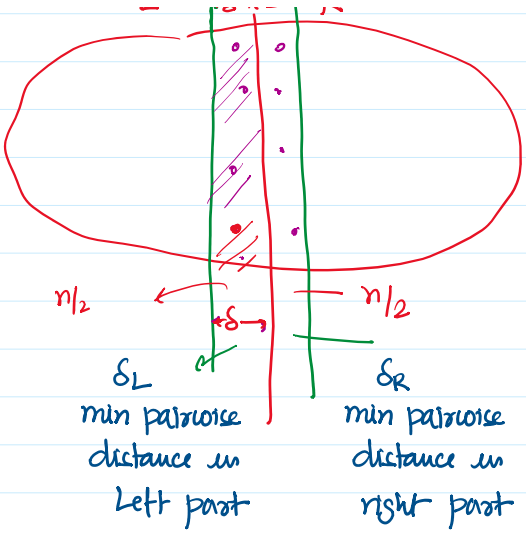
(1) Divide input points into two halves

① Divide input points into two halves

Pick the median based on X-co-ordinate

② Recursively compute δ_L and δ_R

③ return $\min\{\delta_L, \delta_R, \delta\}$



$\delta = \min\{\delta_L, \delta_R\}$

$\left(\frac{n}{2}\right)$ pts may fall in left strip

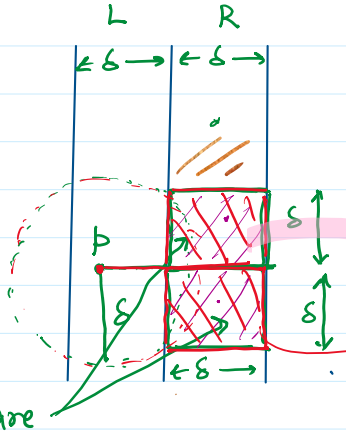
$\left(\frac{n}{2}\right)$ pts may fall in right strip

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) + \underbrace{\left(\frac{n}{2}\right)^2}_{} = O(n^2)$$

Back to $O(n^2)$ complexity

$\delta = \min\{\delta_L, \delta_R\}$

There can't be too many points in the squares



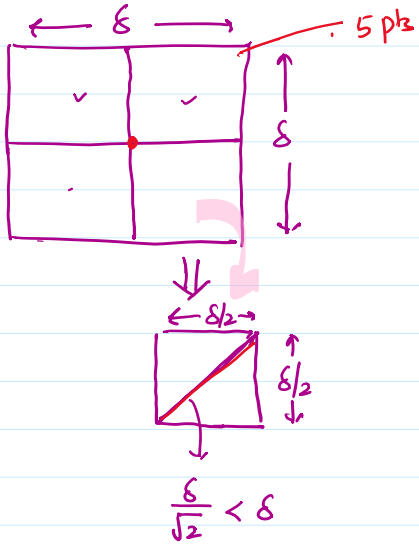
$\delta = \min\{\delta_L, \delta_R\}$

Pigeonhole principle

dist between any pair of points is atleast δ

Que. How many points we can put in square s.t their pairwise distance is atleast δ

Ans: Atmost 4 points in square of size δ

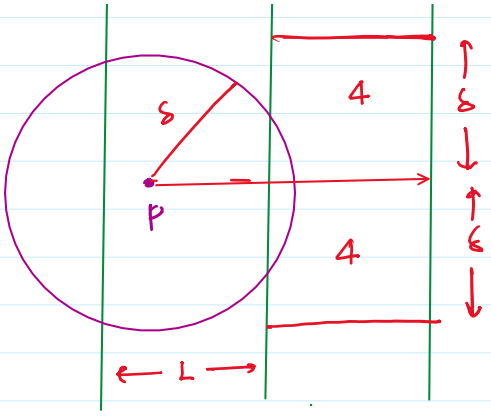


$\frac{\delta}{\sqrt{2}} < \delta$

For each point in the left there are 8 candidate points



4 candidate pts



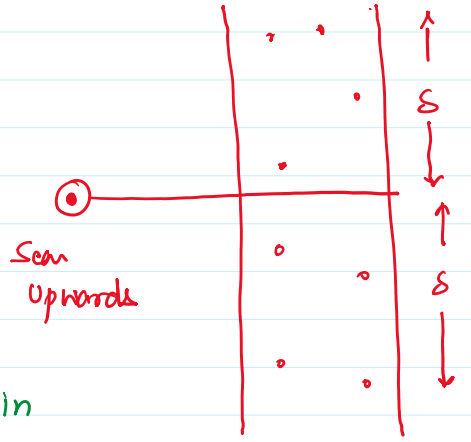
4 candidate pts in each square

Question How to get these 8 points

Sort the points based on Y-co-ordinates

For each point in the left part scan through a window of size 2δ upwards

For each point in left there are atmost 8 candidate points.



This scanning can be done in linear time

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

Sorting based on Y axis, we do it seperately in $O(n \log n)$ time

(1,1) (9,2) (100,0) (1000,5) ⇒ (100,0) (1,1) (9,2) (1000,5)

Step 0: Sort the points based on Y-co-ordinates

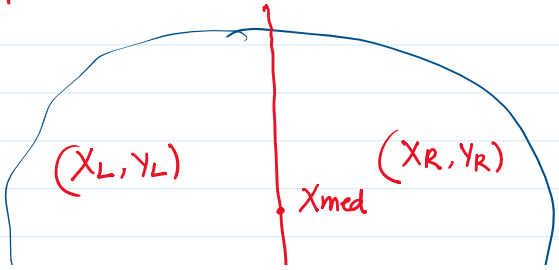
Step 1 X_{med} := median of points sorted based on X-co-ordinates

This splits the input into two halves of size $n/2$ - say X_L and X_R

Split Y into Y_L and Y_R

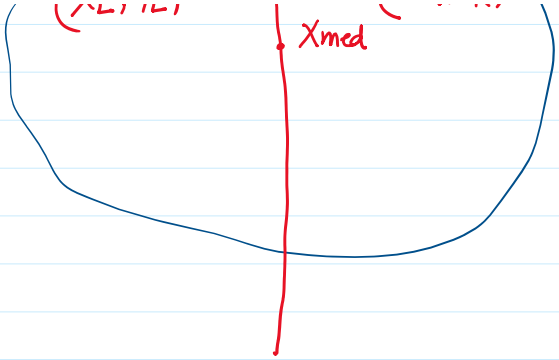
(X_L, Y_L) are pts whose X-co-ordinate is X_L

(X_R, Y_R) " " " " X_R



(X_L, Y_L) are pts whose x-coordinate is X_L

(X_R, Y_R) " " " " X_R



Step2 Recurse in both (X_L, Y_L) and (X_R, Y_R)

δ_L := distance of closest pair in (X_L, Y_L)

δ_R := " " " (X_R, Y_R)

$\delta = \min\{\delta_L, \delta_R\}$

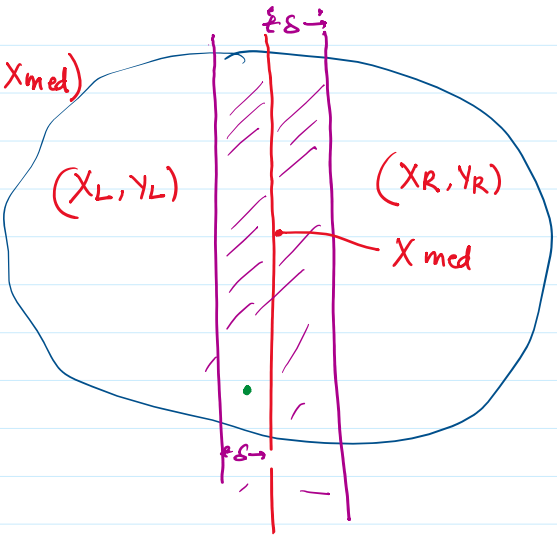
Step4 Putting things together

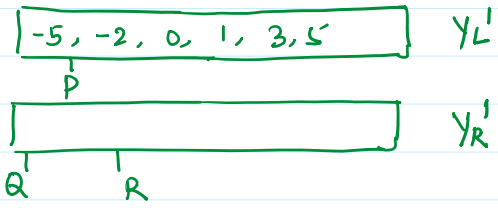
X_L' := pts from X_L whose x-co-ordinates $\in (X_{med} - \delta, X_{med})$

(X_L', Y_L') pts whose X-co-ordinate is X_L'

Similarly compute (X_R', Y_R')

$X_R' \in (X_{med}, X_{med} + \delta)$





$\delta = 2$

maintain 3 ptr

$(3, -5)$

Find pts whose Y axis between -7 to -3

For each point (x_i', y_i'), focus on Y co-ordinate, and find at most 10 pts in the range $y_i' - \delta$, $y_i' + \delta$

$$T(n) = 2T(n/2) + O(n)$$

$O(n)$ for median computation + closest pair across partition

$$= O(n \log n)$$

For sorting pts based on Y-co-ordinates $O(n \log n)$

For sorting plz based on Y-co-ordinates $O(n \log n)$

Overall $O(n \log n)$